HDDA Course Project: Proposal

Team 4: Lev Lesnoi, Dinara Bairamgazieva, Semen Vashurin

Track: Industrial **Direction 5:** PCA that survives outliers

1. What is the question or the problem?

Python now has several PCA implementations that tolerate cellwise outliers (individually corrupted entries) and missing values (robustcov's CellPCA, RobPy's DDC to ROBPCA pipeline, PCP packages), but we found no validation of any of them against the R reference code (cellWise::MacroPCA and the cellPCA authors' code), especially for p >> n and heavy contamination, so practitioners cannot tell which one to trust or where each fail.

2. What shows it is open, or that people need the tool?

1) robustcov 0.2.0 [https://github.com/smiryusupov/robustcov, docs/cellwise_pca.rst, tests/test_cellpca.py]. Its documentation states that numerical equality with the R cellPCA is not claimed (different start, marginal clipping + SVD instead of MacroPCA; different scale estimator). Its 14 CellPCA unit tests contain no comparison with R. The accuracy test uses n = 140, p = 14, 7% bad cells; 2) our check in the setting of the cellPCA paper [https://arxiv.org/pdf/2408.13596, (R code: https://wis.kuleuven.be/statdatascience/robust)] (n = 100, 20% of cells shifted by γσ, two dominant eigenvalues, 25 runs, median angle to the true subspace, γ = 4): at p = 200 CellPCA gives 4.8° (PCA 16.6°), but at p = 20 it gives 32.5° (PCA 26.0°), although the paper reports similar behavior for both p. PCP stays at 10.7°. R not yet run: a lead, not a verdict (script attached); 3) context. scikit-learn issue #5851 "Robust PCA" (closed) asked for PCP-type methods; PR #6806 stalled with transform on new samples unresolved, and the FAQ allows external packages [https://github.com/scikit-learn/scikit-learn/issues/5851, https://github.com/scikit-learn/scikit-learn/pull/6806]. RobPy exists because Python lacked robust methods [https://arxiv.org/pdf/2411.01954]. A 2026 review calls the cellwise approach increasingly dominant for high-dimensional data [https://arxiv.org/pdf/2604.14182].

3. What already exists, and why is it not enough?

1) RobPy: ROBPCA handles whole-row outliers only; DDC detects and imputes cells, so the manual DDC to ROBPCA two-step works (comparable to CellPCA in our runs at p ≤ 200), but DDC alone took 13 seconds at p = 100 and 235 seconds at p = 500 (single runs) and, unlike R, has no fastDDC option [https://arxiv.org/pdf/1712.05151, cran.r-project.org/package=cellwise]; 2) robustcov: CellPCA, CellRCov, PCP, scikit-learn-style API, fast (2 seconds on 72 * 7129) but unvalidated against R; API renames in progress; 3) PCP packages (dfm/pcp, robust-pca, rpcaADMM, skpcp) fit a different model, cannot project new samples (the obstacle in #5851) and are mostly scripts. R (cellWise::MacroPCA [https://arxiv.org/pdf/1806.00954, https://cran.r-project.org/package=cellwise], GPL ≥ 2; cellPCA code from its authors [https://arxiv.org/pdf/2408.13596]) is the reference, but nothing connects it to the Python tools.

What we add: a pip-installable suite that puts each Python method behind one scikit-learn-style adapter; runs the R reference as an optional, copying no R code; generates the published scenarios plus our own grid; tests conformance with tolerances fixed in advance; outputs a map of failure regimes and minimal reproducible reports for maintainers.

4. Which data will you use?

1) Golub leukemia gene expression: 72 samples * 7129 genes, https://www.openml.org/d/45090, with injected outliers; 2) RobPy bundled data: glass spectra 180 * 750 (p > n); TopGear 297 cars * 13 numeric variables, 110 missing cells, documented cellwise anomalies [https://arxiv.org/pdf/2411.01954]; 3) Synthetic: a) the setting of [https://arxiv.org/pdf/2408.13596] (n = 100, p ∈ {20, 200}, 20% cells, γ ∈ [0, 6], row-wise and mixed contamination), plus a variant with 10% of cells set to missing; b) our grid n ∈ {60, 100, 200}, p ∈ {20, 100, 200, 1000, 7129}, 1-20% cells, three magnitudes, 25 runs per cell.

5. What will you compare against?

Method	Role
R cellWise::MacroPCA; cellPCA R code	Reference for conformance (not a competitor)
robustcov CellPCA and PCP; dfm/pcp	Systems under test (new, unvalidated; two PCP codes cross-check each other)
RobPy DDC to ROBPCA (manual two-step)	System under test; strongest existing RobPy pipeline
RobPy ROBPCA, spherical PCA; scikit-learn PCA	Row-wise robust baselines and lower bound

6. How will you measure success?

Metrics: largest principal angle to the true and to the R-reference subspace; cell-flag precision/recall/F1; imputation RMSE; run time versus p; on real data, the angle between subspaces fitted before and after outlier injection (ALL/AML classification (acute lymphoblastic leukemia/ acute myelogenous leukemia) is too easy to separate methods). Criteria, fixed in advance: 1) the R reference reproduces the behavior reported in [https://arxiv.org/pdf/2408.13596] (error ≥ x3 below PCA at γ = 6 for p = 20 and 200); otherwise, our setup is wrong; 2) a Python method "agrees" with R in a scenario if its median angle to the R subspace is ≤ max(1°, 95th percentile of R vs R angles over 25 seeds); each disagreement is re-run on ≥ 2 generators; 3) a failure map over ≥ 3 values of p/n * 5 contamination levels * 3 magnitudes and a reproducible report to the maintainers for every disagreement (or a documented agreement); 4) a pip-installable package with ≥ 80% test coverage, documentation and a worked TopGear and Golub example, running in ≤ 30 min on a laptop without R.

7. What is the main risk, and what will you do if it happens?

Main risk is the R reference cannot be run reliably (installation, MacroPCA slow at large p without fastDDC, or the cellPCA code behind the link in [https://arxiv.org/pdf/2408.13596] unavailable). Plan: install R and reproduce the TopGear example from the cellWise vignette; use fastDDC and limit R runs to p ≤ 1000 if needed; without cellPCA code, MacroPCA alone is the reference and cellPCA is checked against the behaviour reported in the paper. If the discrepancy is an artefact (R behaves the same), the finding becomes "robustcov agrees with R", still a deliverable together with the suite and map.